AI ASSISTED CODING

LAB-15.5

Name : Rishik

Hall Ticket No: 2303a51778

Batch:12

Task 1 – Student Attendance API Task:

Create a RESTful API for student attendance tracking.

Instructions:

- Endpoints required:

o GET /attendance → View attendance records

o POST /attendance → Mark attendance

o PUT /attendance/{id} → Update attendance record

o DELETE /attendance/{id} → Remove attendance entry

- Ensure JSON-based request and response handling.

Expected Output:

- API for maintaining and updating attendance records.

PROMPT:

"""Generate a Python Flask REST API for student attendance with in-memory storage.

Implement GET /attendance, POST /attendance, PUT /attendance/<id>, DELETE /attendance/<id>.

Use JSON request/response and add comments."""

GIVEN CODE:

app.py > get_attendance

```python
from flask import Flask, request, jsonify

app = Flask(__name__)

# In-memory storage for attendance records
attendance_records = [
    {"id": 1, "student_name": "Alice", "status": "Present"},
    {"id": 2, "student_name": "Bob", "status": "Absent"}
]

# Counter for generating new IDs
next_id = 3

# GET all attendance records
@app.route('/attendance', methods=['GET'])
def get_attendance():
    return jsonify(attendance_records), 200

# POST a new attendance record
@app.route('/attendance', methods=['POST'])
def add_attendance():
    global next_id
    data = request.get_json()

    # Validate required fields
    if not data or not data.get('student_name') or not data.get('status'):
        return jsonify({"error": "Missing required fields"}), 400

    # Create new record
    new_record = {
        "id": next_id,
        "student_name": data['student_name'],
        "status": data['status']
    }
    attendance_records.append(new_record)
    next_id += 1

    return jsonify(new_record), 201

# PUT to update an attendance record
@app.route('/attendance/<int:id>', methods=['PUT'])
def update_attendance(id):
```

OUTPUT:

http://127.0.0.1:5000/attendance

Pretty-print


```
[
  {
    "id": 1,
    "status": "Present",
    "student_name": "Alice"
  },
  {
    "id": 2,
    "status": "Absent",
    "student_name": "Bob"
  }
]
```


EXPLANATION:

In this task, I understood how to create a simple attendance system using Flask API. I learned how to add, view, update, and delete records using different methods like GET, POST, PUT, and DELETE. I also understood how JSON is used to send data and how the server handles requests. This helped me understand how backend systems work in real applications.

Task 2 – Inventory Management API

Task:

Develop a REST API for warehouse inventory management.

Instructions:

- Endpoints required:

o GET /inventory → List all items

o POST /inventory → Add new item

o PUT /inventory/{id} → Update stock quantity

o DELETE /inventory/{id} → Remove an item

- Implement validation for stock values.

Expected Output:

- API capable of managing inventory stock levels.

PROMPT:

"""Create a Flask REST API for inventory management with item id, name, and quantity. Implement GET, POST, PUT, DELETE endpoints for /inventory.
Add validation so stock quantity cannot be negative."""

CODE:

```
app.py > ...
from flask import Flask, request, jsonify

app = Flask(__name__)

# In-memory storage for inventory items
inventory = {}
next_id = 1

@app.route('/inventory', methods=['GET'])
def get_inventory():
    """Return all items in inventory"""
    return jsonify(list(inventory.values()))

@app.route('/inventory', methods=['POST'])
def add_item():
    """Add a new item to inventory"""
    data = request.get_json()

    # Validate input
    if not data or 'name' not in data or 'quantity' not in data:
        return jsonify({'error': 'Missing name or quantity'}), 400

    if data['quantity'] < 0:
        return jsonify({'error': 'Quantity cannot be negative'}), 400

    global next_id
    item = {
        'id': next_id,
        'name': data['name'],
        'quantity': data['quantity']
    }
    inventory[next_id] = item
    next_id += 1

    return jsonify(item), 201

@app.route('/inventory/<int:item_id>', methods=['PUT'])
def update_item(item_id):
    """Update item quantity"""
    if item_id not in inventory:
        return jsonify({'error': 'Item not found'}), 404
```

OUTPUT:

http://127.0.0.1:5000/inventory

Pretty-print


```
[
  {
    "id": 1,
    "name": "Laptop",
    "quantity": 5
  }
]
```


EXPLANATION:

In this task, I understood how to create an inventory management system using Flask API. I learned how to add, view, update, and delete items using different HTTP methods. I also understood how to validate data like quantity and handle requests using JSON. This task helped me understand how real-world applications manage and update data dynamically.

Task 3 – Hotel Room Booking API

Task:

Develop a RESTful API for hotel room booking.

Instructions:

• Endpoints required:

o GET /rooms → View available rooms

o POST /rooms/book → Book a room

o GET /rooms/{id} → View booking details

o DELETE /rooms/{id} → Cancel booking

• Implement error handling for unavailable rooms.

Expected Output:

• API simulating hotel room booking operations.

PROMPT:

Generate a Python Flask REST API for hotel room booking.
Requirements:
- Use in-memory storage (no database)
- Each room should have: id, room_number, and status (Available/Booked)
- Implement the following endpoints:
- GET /rooms → view all rooms
- POST /rooms/book → book a room
- GET /rooms/<id> → view booking details
- DELETE /rooms/<id> → cancel booking
- Add error handling:
- If a room is already booked, return an error message
- If room not found, return 404 error
- Use JSON format for request and response
- Write beginner-friendly code with simple comments

CODE:

app.py > ...

```python
from flask import Flask, request, jsonify

app = Flask(__name__)

# In-memory storage for rooms
rooms = [
    {"id": 1, "room_number": 101, "status": "Available"},
    {"id": 2, "room_number": 102, "status": "Available"},
    {"id": 3, "room_number": 103, "status": "Booked"},
]

# GET all rooms
@app.route('/rooms', methods=['GET'])
def get_all_rooms():
    return jsonify(rooms), 200

# POST book a room
@app.route('/rooms/book', methods=['POST'])
def book_room():
    data = request.json
    room_id = data.get('id')

    # Find the room
    room = next((r for r in rooms if r["id"] == room_id), None)

    if not room:
```

ROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
S C:\Users\Rishik\OneDrive\Desktop\ass15.5> & C:/Users/Rishik/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/U
op/ass15.5/app.py
* Serving Flask app 'app'
* Debug mode: on
ARNING: This is a development server. Do no[illegible]tion deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
ress CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 311-037-698
27.0.0.1 - - [21/Mar/2026 11:31:45] "GET /rooms HTTP/1.1" 200 -
```

Follow link (ctrl + click)

OUTPUT:

http://127.0.0.1:5000/rooms

Pretty-print


```
[
  {
    "id": 1,
    "room_number": 101,
    "status": "Available"
  },
  {
    "id": 2,
    "room_number": 102,
    "status": "Available"
  },
  {
    "id": 3,
    "room_number": 103,
    "status": "Booked"
  }
]
```


Task 4 – Online Voting API

Task:

Create a RESTful API for an online voting system. Instructions:

• Endpoints required:

o GET /candidates → List candidates

o POST /vote → Cast a vote

o GET /results → View voting results

• Ensure one vote per user using in-memory validation.

Expected Output

API simulating a secure online voting process.

PROMPT:

Generate a Python Flask REST API for student marks management.

Requirements:

- Use in-memory storage (no database)
- Each student record should have: id, name, and marks
- Implement the following endpoints:
  - GET /students → view all student records
  - POST /students → add a new student
  - PUT /students/<id> → update student marks
  - DELETE /students/<id> → delete a student record
- Add validation:
  - Marks should be between 0 and 100
- Return proper error messages if student not found
- Use JSON format for request and response

GIVEN OUTPUT:

app.py task3

app.py > add_student

```python
from flask import Flask, request, jsonify

app = Flask(__name__)

# In-memory storage for students
students = {}
next_id = 1

# GET all students
@app.route('/students', methods=['GET'])
def get_students():
    return jsonify(list(students.values()))

# POST a new student
@app.route('/students', methods=['POST'])
def add_student():
    global next_id
    data = request.json

    # Validate marks
    if 'marks' in data and (data['marks'] < 0 or data['marks'] > 100):
        return jsonify({'error': 'Marks must be between 0 and 100'}), 400

    student = {
        'id': next_id,
        'name': data.get('name'),
        'marks': data.get('marks', 0)
    }
    students[next_id] = student
    next_id += 1
    return jsonify(student), 201
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 311-037-698
127.0.0.1 - - [21/Mar/2026 11:47:20] "GET /students HTTP/1.1" 200 -
```

OUTPUT:

http://127.0.0.1:5000/students

Pretty-print


```
[
  {
    "id": 1,
    "marks": 85,
    "name": "Rishik"
  }
]
```

EXPLANATION:

In this task, I understood how to create a student marks management system using Flask API. I learned how to add, view, update, and delete student records using different HTTP methods. I also understood how to validate marks and handle data using JSON. This task helped me understand how data is managed and updated in backend applications.

Task 5 – Teacher Management API
Task:
Develop a RESTful API to manage teacher records.
Instructions:
• Endpoints required:
o GET /teachers → List all teachers
o POST /teachers → Add a new teacher
o PUT /teachers/{id} → Update teacher details
o DELETE /teachers/{id} → Remove a teacher
• Ensure proper error handling and JSON responses.
Expected Output:
• Functional API for managing teacher information.

PROMPT:
Generate a Python Flask REST API for user authentication.
Requirements:

- Use in-memory storage (no database)
- Each user should have: id, username, and password
- Implement the following endpoints:
    - POST /register → register a new user
    - POST /login → authenticate user login
    - GET /users → view all registered users
    - Username must be unique
    - Password should not be empty
- Return proper error messages for invalid login or duplicate users
- Use JSON format for request and response

GIVEN OUTPUT:

ass.py > register

```python
from flask import Flask, request, jsonify

app = Flask(__name__)

# In-memory storage for users
users = []
user_id_counter = 1

@app.route('/register', methods=['POST'])
def register():
    """Register a new user"""
    global user_id_counter
    data = request.get_json()

    # Validate inputs
    if not data or not data.get('username') or not data.get('password'):
        return jsonify({'error': 'Username and password required'}), 400

    # Check if username already exists
    if any(user['username'] == data['username'] for user in users):
        return jsonify({'error': 'Username already exists'}), 409

    # Create new user
    new_user = {
        'id': user_id_counter,
        'username': data['username'],
        'password': data['password']
    }
    users.append(new_user)
    user_id_counter += 1

```

OBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
C:\Users\Rishik\OneDrive\Desktop\ass15.5> & C:/Users/Rishik/AppData/Local/Python/pythoncore
p/ass15.5/ass.py
7.0.0.1 - - [22/Mar/2026 13:30:27] "GET /register HTTP/1.1" 405 -
7.0.0.1 - - [22/Mar/2026 13:30:42] "GET /register HTTP/1.1" 405 -
7.0.0.1 - - [22/Mar/2026 13:30:46] "GET /register HTTP/1.1" 405 -
7.0.0.1 - - [22/Mar/2026 13:30:47] "GET /register HTTP/1.1" 405 -
7.0.0.1 - - [22/Mar/2026 13:32:36] "POST /register HTTP/1.1" 201 -
7.0.0.1 - - [22/Mar/2026 13:32:53] "GET /users HTTP/1.1" 200 -
```

OUTPUT:

Pretty-print


```
{
  "users": [
    {
      "id": 1,
      "password": "1234",
      "username": "rishik"
    }
  ]
}
```

EXPLANATION:

In this task, I understood how to create a user authentication system using Flask API. I learned how to register new users and handle login functionality using POST requests. I also understood how to validate user data and store it in memory. This task helped me understand how authentication works in real applications.